WEEK 2 · LESSON 9 OF 24 · HANDOUT

Boosting: Learning from Mistakes

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=3CC4N4z3GJc — open the lesson online to watch inside the course.

🎓 **Go deeper:** StatQuest: AdaBoost, Clearly Explained — https://www.youtube.com/watch?v=LsK-xG1cLYA

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 2 (scikit-learn core developers) — https://www.youtube.com/watch?v=gK43gtGh49o

Your goal in this lesson: Explain how boosting builds models one after another to fix earlier mistakes, train gradient boosting in scikit-learn, and tune its learning rate and number of trees.

Bagging vs boosting

A random forest trains its trees **side by side**, independently. **Boosting** trains them **one after another**: each new tree focuses on what the previous trees got wrong. It is like a student who, after every practice test, studies only the questions they missed.

	Bagging (random forest)	Boosting
Trees built	In parallel, independently	One after another
Each tree	Deep, strong	Small, weak (often 2–3 levels)
Main effect	Reduces variance (overfitting)	Reduces bias (underfitting)
Risk	Little	Can overfit if it runs too long

Gradient boosting in one picture

For a number like rent:

1. Start with a simple guess: the average rent.
2. Compute the **residuals** — the errors (real − predicted) for every home.
3. Train a small tree to predict those **errors**.
4. Add a **small fraction** of that tree's prediction to the model (the **learning rate**, for example 0.1).
5. Repeat hundreds of times.

Each step moves the predictions a little closer to the truth — the same "small steps downhill" idea as gradient descent in Math & Python for AI. For classification the same idea works on the log-odds of the probability.

See it in code

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(random_state=42)    # 100 trees of depth 3, learning rate 0.1
gb.fit(X_train, y_train)
print("Train:", round(gb.score(X_train, y_train), 3))
print("Test: ", round(gb.score(X_test, y_test), 3))
```

You should see about **0.90** on training data and **0.84** on the test set.

The two settings that matter most

```
for lr in [0.05, 0.1, 0.3]:
    m = GradientBoostingClassifier(learning_rate=lr, n_estimators=200, max_depth=2,
                                   random_state=42).fit(X_train, y_train)
    print("learning rate", lr, "test", round(m.score(X_test, y_test), 3))
```

- **learning_rate** — how big each correction is. Small values (0.05) learn slowly but carefully; big values (0.3) rush and overfit.
- **n_estimators** — how many trees. With a small learning rate you need more trees.
- **max_depth** — keep trees small (2–4): each one should be a weak learner.

A rule used by professionals: **lower the learning rate, raise the number of trees**, and stop when the validation score stops improving. `GradientBoostingClassifier(n_iter_no_change=10, validation_fraction=0.1)` does this early stopping for you.

Watch it learn

```
import numpy as np
import matplotlib.pyplot as plt

m = GradientBoostingClassifier(learning_rate=0.1, n_estimators=300, max_depth=2,
                               random_state=42).fit(X_train, y_train)
test_acc = [np.mean(p == y_test) for p in m.staged_predict(X_test)]
train_acc = [np.mean(p == y_train) for p in m.staged_predict(X_train)]
plt.plot(train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("number of trees")
plt.ylabel("accuracy")
plt.legend()
plt.show()
print("Best number of trees on test:", int(np.argmax(test_acc)) + 1)
```

Training accuracy keeps rising; test accuracy rises, then flattens. The gap that opens is overfitting.

AdaBoost and the famous libraries

- **AdaBoost** (1990s; it powered the first real-time face detectors in 2001) was the first popular boosting method: it gives **more weight** to misclassified examples so the next model concentrates on them. `AdaBoostClassifier` in scikit-learn.
- **HistGradientBoostingClassifier** — scikit-learn's fast version for large data; it also handles missing values.
- **XGBoost, LightGBM and CatBoost** — separate libraries that win many data competitions on tables of data. They follow the same fit/predict pattern.

On this small, simple dataset, boosting is about as good as logistic regression — the loans were simulated with a fairly simple rule. On real, messy tables, gradient boosting is very often the most accurate model.

Key terms

- **Boosting** — training models in sequence, each fixing the errors of the previous ones.
- **Weak learner** — a small, simple model (a shallow tree).
- **Residual** — real value minus predicted value.
- **Learning rate (shrinkage)** — how much of each new tree is added.
- **Early stopping** — stop adding trees when the validation score stops improving.

Remember this: Boosting adds many small trees one by one, each correcting the last; a small learning rate with enough trees and early stopping gives some of the most accurate models for tables.

Name: _____ Date: _____

Activity

Task 1

Train `HistGradientBoostingClassifier(random_state=42)` and compare its test accuracy and speed with `GradientBoostingClassifier`.

Task 2

Train `AdaBoostClassifier(n_estimators=200, random_state=42)`. How does it compare?

Task 3

Use early stopping: `GradientBoostingClassifier(n_estimators=1000, learning_rate=0.05, n_iter_no_change=10, random_state=42)`. How many trees did it actually build? (Look at `n_estimators_`.)

Task 4

In two sentences, explain the difference between bagging and boosting to a classmate.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does boosting build its trees?
   - A. By copying one tree
   - B. All at the same time, independently
   - C. One after another, each fixing earlier mistakes
   - D. At random, without data
2. In gradient boosting for regression, what does each new tree learn to predict?
   - A. The features
   - B. The residuals (errors) of the current model
   - C. The original target
   - D. The learning rate
3. What does a smaller learning rate usually need?
   - A. No trees
   - B. More trees
   - C. Scaled features
   - D. Fewer trees
4. Why are boosting trees usually kept shallow (depth 2–4)?
   - A. Shallow trees need scaling
   - B. Each one should be a weak learner adding a small correction
   - C. Deep trees are not allowed
   - D. To make the forest bigger
5. What is early stopping?
   - A. Train for one tree only
   - B. Stop training when the validation score stops improving
   - C. Stop when accuracy is 100%
   - D. Delete the test set
6. Which libraries are famous gradient-boosting tools?
   - A. NumPy and Pandas
   - B. XGBoost, LightGBM and CatBoost
   - C. Matplotlib and Seaborn
   - D. Flask and Django

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.